

# Trabalho 03

## Grafos

Gabriel Leite Fonseca e César Augusto Bispo Santos

24/06/2026

---

### Seção 01

[ADICIONAR]

---

### Seção 02

#### 2.1 - Estrutura utilizada para representação do grafo

Para a implementação da biblioteca foi utilizada a representação por **lista de adjacência**. Nessa abordagem, cada vértice mantém uma lista contendo as arestas conectadas a ele.

A escolha dessa estrutura ocorreu por sua simplicidade de implementação e por utilizar memória apenas para as conexões existentes no grafo. Dessa forma, não é necessário armazenar todas as combinações possíveis entre vértices, como ocorreria em uma matriz de adjacência.

A estrutura principal do grafo foi implementada na classe `Graph<T>`.

```
private List<Vertex<T>> vertices;  
private Map<Vertex<T>, List<Edge<T>>> adjacencyList;
```

A lista `vertices` é responsável por armazenar todos os vértices do grafo, enquanto `adjacencyList` mantém as conexões existentes entre eles.

Os vértices são representados pela classe `Vertex<T>`, permitindo armazenar qualquer tipo de dado através do uso de generics.

```
public class Vertex<T> {  
    private T value;  
}
```

Já as arestas são representadas pela classe `Edge<T>`, responsável por armazenar a origem, o destino e a capacidade associada à conexão.

```
public class Edge<T> {  
  
    private Vertex<T> origin;  
    private Vertex<T> destination;  
    private float capacity;  
  
}
```

Essa estrutura serviu como base para a implementação dos algoritmos desenvolvidos ao longo do trabalho.

## 2.2 - Inserção de vértices

A inserção de vértices é realizada pelo método `addVertex()`. Antes da criação de um novo vértice, a implementação percorre a lista de vértices existentes para verificar se o valor informado já está presente no grafo.

```
public void addVertex(T value) {  
  
    for (Vertex<T> vertex : vertices) {  
        if (vertex.getValue().equals(value)) {  
            return;  
        }  
    }  
  
    Vertex<T> vertex = new Vertex<>(value);  
  
    vertices.add(vertex);  
    adjacencyList.put(vertex, new ArrayList<>());  
}
```

A verificação evita a criação de vértices duplicados e garante que cada valor esteja associado a apenas um vértice.

Como a implementação pode percorrer toda a lista de vértices no pior caso, sua complexidade é  $O(V)$ , onde  $V$  representa a quantidade de vértices do grafo.

## 2.3 - Inserção de arestas

A criação de conexões entre vértices é realizada pelo método `addEdge()`.

```
public void addEdge(  
    T origin,  
    T destination,  
    float capacity  
) {  
    ...  
    adjacencyList.get(originVertex)  
        .add(  
        new Edge<>(  
            originVertex,  
            destinationVertex,  
            capacity  
        )  
    );  
}
```

Inicialmente são localizados os vértices de origem e destino informados pelo usuário. Após a localização, uma nova aresta é criada e adicionada à lista de adjacência do vértice de origem.

Como a implementação precisa localizar os vértices antes de criar a aresta, a complexidade da operação é  $O(V)$  no pior caso.

## 2.4 - Busca em Largura (BFS)

Para realizar o caminhamento do grafo foi implementado o algoritmo de Busca em Largura (Breadth-First Search - BFS). A implementação utiliza uma fila para controlar a ordem de visita dos vértices.

```
Queue<Vertex<T>> queue = new LinkedList<>();  
List<Vertex<T>> visited = new ArrayList<>();  
  
queue.add(start);  
visited.add(start);  
  
while (!queue.isEmpty()) {  
    Vertex<T> current = queue.poll();
```

```

    for (Edge<T> edge : adjacencyList.get(current)) {

        Vertex<T> neighbor =
            edge.getDestination();

        if (!visited.contains(neighbor)) {
            visited.add(neighbor);
            queue.add(neighbor);
        }
    }
}

```

O algoritmo visita inicialmente o vértice de origem e, em seguida, explora todos os seus vizinhos antes de avançar para os próximos níveis do grafo.

A utilização da lista de adjacência permite acessar diretamente os vértices conectados ao vértice atual, tornando a navegação eficiente.

Cada vértice é visitado apenas uma vez e cada aresta é analisada no máximo uma vez durante a execução. Dessa forma, a complexidade do algoritmo é  $O(V + E)$ , onde:

- V representa a quantidade de vértices;
- E representa a quantidade de arestas.

## 2.5 - Algoritmo de Fluxo Máximo (Ford-Fulkerson)

Para a resolução do problema prático foi implementado o algoritmo de Ford-Fulkerson em uma classe separada da biblioteca base.

A separação entre estrutura e algoritmo foi realizada para manter a biblioteca genérica e reutilizável, permitindo que outros algoritmos possam ser implementados futuramente sem alterações na estrutura do grafo.

O algoritmo é responsável por determinar o fluxo máximo que pode ser transportado entre uma origem e um destino respeitando a capacidade das arestas.

A busca por caminhos é realizada através de uma busca em profundidade (DFS).

```

private boolean dfsPath(
    Graph<T> graph,
    Vertex<T> current,
    Vertex<T> sink,
    List<Edge<T>> path,

```

```
        List<Vertex<T>> visited  
    )
```

Após encontrar um caminho válido, é determinado o gargalo do percurso, correspondente à menor capacidade encontrada entre as arestas.

```
float bottleneck = Float.MAX_VALUE;  
  
for (Edge<T> edge : path) {  
    bottleneck = Math.min(  
        bottleneck,  
        edge.getCapacity()  
    );  
}
```

Em seguida, as capacidades são atualizadas e o fluxo encontrado é somado ao fluxo total da rede.

O processo continua até que não existam mais caminhos possíveis entre origem e destino.

Como a implementação realiza sucessivas buscas por caminhos aumentantes, sua complexidade é  $O(E \times F)$ , onde:

- E representa a quantidade de arestas do grafo;
- F representa o valor do fluxo máximo encontrado.

Isso ocorre porque, a cada aumento de fluxo, é necessário percorrer arestas da rede para encontrar um novo caminho válido até que não seja mais possível aumentar o fluxo total.

## Seção 03

[ADICIONAR]

---

## Seção 04

### 4.1 Responsabilidades atividades

| Atividade                                                    | Responsável                |
|--------------------------------------------------------------|----------------------------|
| Implementação da classe <b>Vertex</b>                        | Gabriel Leite Fonseca      |
| Implementação da classe <b>Edge</b>                          | Gabriel Leite Fonseca      |
| Implementação da classe <b>Graph</b>                         | Gabriel Leite Fonseca      |
| Implementação dos métodos de inserção de vértices e arestas  | Gabriel Leite Fonseca      |
| Implementação do algoritmo de Busca em Largura (BFS)         | Gabriel Leite Fonseca      |
| Elaboração da Seção 2 do relatório                           | Gabriel Leite Fonseca      |
| Implementação do algoritmo de Ford-Fulkerson                 | Gabriel Leite Fonseca      |
| Implementação do algoritmo de Ford-Fulkerson                 | César Augusto Bispo Santos |
| Desenvolvimento das funcionalidades específicas da aplicação | César Augusto Bispo Santos |
| Integração da biblioteca ao problema proposto                | César Augusto Bispo Santos |
| Elaboração das Seções 3, 4 e demais partes do relatório      | César Augusto Bispo Santos |
| Testes e validação da aplicação                              | César Augusto Bispo Santos |

## 4.2 Repositório do Projeto

github: <https://github.com/Gabedf/TPA/tree/main/Graphs/>

## 4.3 Utilização de Ferramentas de Inteligência Artificial

Durante o desenvolvimento do trabalho foram utilizadas ferramentas baseadas em Large Language Models (LLMs) como apoio à pesquisa, implementação e documentação do projeto.

Essas ferramentas foram empregadas para esclarecer dúvidas sobre conceitos de grafos, estruturas de dados da linguagem Java e possíveis abordagens de implementação. Também foram utilizadas para auxiliar na elaboração de trechos do relatório e na revisão de partes do código.

O desenvolvimento ocorreu de forma iterativa, utilizando as sugestões fornecidas como ponto de partida para discussão e implementação das soluções. Ao longo do processo foram realizados testes, correções e adaptações para adequar o código aos requisitos definidos pelo trabalho.